

# Corrigiendo `search_best_beta()`

Errores metodológicos encontrados y una búsqueda más precisa, sin encarecerla

Walter Quiñonez

Reporte generado con Claude Code · 9 de septiembre de 2026

*Se analizó `MemDNN.search_best_beta()` (`MemCrossbarClass_beta_por_capa.py`, líneas 268-370), se encontraron cinco errores/carencias metodológicas, y se implementó una versión corregida como subclase (`MemDNNMejorado`) que no modifica ningún archivo original. Validada empíricamente contra el ground truth real: reduce el regret en el peor caso de 20.3pp a 0.45pp, sin encarecer significativamente el cómputo.*

## Índice

|                                                                                   |   |
|-----------------------------------------------------------------------------------|---|
| 1. Errores encontrados en el método original                                      | 1 |
| 2. La corrección clave: centrar la búsqueda en la fórmula analítica ya encontrada | 1 |
| 3. Validación empírica: original vs. mejorado                                     | 2 |
| 4. Cómo usarlo                                                                    | 2 |
| 5. Limitaciones                                                                   | 3 |

## 1. Errores encontrados en el método original

1. **Bug:** todo el cuerpo está adentro de `if a <= 10000:` sin `else` – para `a_pot > 10000` (curva P/D válida, más lineal) la función no hace nada y devuelve `None` en silencio.
2. **1 sola época** de entrenamiento por candidato de `beta` – el estudio de `analisis_epochs_series_minimos/` (ground truth de 100 épocas) midió hasta 1.48pp de regret con 1 época, y <1pp con 8.
3. **Sin control de aleatoriedad entre betas** (el más serio): cada candidato usa una inicialización de conductancias y un orden de batches completamente independientes de los demás, contaminando la comparación con ruido que no tiene que ver con `beta`.
4. **Grilla mal elegida y ciega:** la llamada real usa `beta_max=300`, `points=2` – literalmente prueba solo  $\beta \in \{1, 300\}$  – y el rango se queda corto (el ground truth encuentra óptimos de hasta 750).
5. **Código muerto:** el parámetro `std_range` nunca se usa.

## 2. La corrección clave: centrar la búsqueda en la fórmula analítica ya encontrada

En `formula_beta_optimo_propuesta/` ya se había ajustado (18 curvas,  $R^2 = 0.938$ ):

$$\beta_{\text{óptimo}} \approx 1.34 \times 10^{-11} \cdot \sigma_W^{-3.05} \cdot \text{paso\_medio}^{-0.61}$$

Se la validó de nuevo contra las 30 combinaciones INL×pulsos hoy disponibles (12 más que las usadas para ajustarla):

| Cociente $\beta_{\text{predicho}}/\beta_{\text{óptimo real}}$ | valor        |
|---------------------------------------------------------------|--------------|
| mediana                                                       | 0.948        |
| mínimo                                                        | 0.720        |
| máximo                                                        | 1.261        |
| % de los 30 casos dentro de un factor 2×                      | <b>100 %</b> |

Con una predicción tan buena, la búsqueda ya no necesita cubrir  $[1, 2000]$  a ciegas: se centra en `beta_pred` (calculado en milisegundos) con un margen 4× (con colchón sobre el error máximo observado de 1.26×), y expande automáticamente si el óptimo cae en el borde de esa grilla angosta (red de seguridad ante regímenes donde la fórmula falle).

## 3. Validación empírica: original vs. mejorado

2 combos representativos (extremos de no linealidad), 5 repeticiones con semillas nuevas cada uno, comparando contra el ground truth completo (100 épocas, 20 series, 5 folds):

| Método                             | regret medio   | regret mediana | <b>regret peor caso</b> | tiempo prom. |
|------------------------------------|----------------|----------------|-------------------------|--------------|
| (a) original (como se usa hoy)     | 10.19 pp       | 10.19 pp       | <b>20.30 pp</b>         | 11.8 s       |
| (b) original (defaults de fábrica) | 0.24 pp        | 0.03 pp        | 2.16 pp                 | 118.2 s      |
| <b>(c) mejorado</b>                | <b>0.08 pp</b> | <b>0.04 pp</b> | <b>0.45 pp</b>          | 218.2 s      |

Cuadro 1: 10 corridas por método (2 combos × 5 repeticiones), semillas nuevas no reutilizadas del grid search original.

El método que corre hoy en producción (a) elige siempre  $\beta = 1$  en el combo más no lineal (INL= $2 \times 10^{-1}$ , óptimo real=100), perdiendo sistemáticamente  $\sim 20$  puntos porcentuales de accuracy – con solo 2 puntos de grilla, si ninguno cae cerca del óptimo no hay forma de recuperarse. El mejorado nunca superó 0.45pp de regret en las 10 corridas, a un costo de  $\sim 3.6$  minutos por búsqueda – despreciable frente a un entrenamiento completo real (horas).

## 4. Cómo usarlo

```
from search_best_beta_mejorado import MemDNNMejorado

model = MemDNNMejorado(sizes=sizes, beta=beta_inicial, pot=pot, dep=dep,
                        G0_distribution=G0_distrbtn, fixed=fixed, device=device)
best_beta, best_acc, info = model.search_best_beta(
```

```
train_loaders, val_loaders, criterion, a_pot, lr,  
delta_t_forward, delta_t_pulse, Vr, Vs)
```

Reemplazo “drop-in”: no se modificó `MemCrossbarClass_beta_por_capa.py` ni `Crossbar_train_experimento_beta_por_capa.py`; `MemDNNMejorado` hereda todo de `MemDNN` y solo sobrescribe `search_best_beta`.

## 5. Limitaciones

- La fórmula que centra la grilla fue validada solo para `sizes=[784,10]` y ratio  $G_{max}/G_{min}=10$ ; en otro régimen, la red de seguridad (expansión automática) es la que sostiene la precisión.
- Ni la fórmula ni la búsqueda optimizan betas distintos por capa (mismo alcance que el original).
- Validación empírica con 2 combos y 5 repeticiones (no los 30 combos completos), por costo de tiempo.

Ver `documentacion_mejora_search_best_beta.md` en esta misma carpeta para el desarrollo completo, código y tablas verificables por un humano.